

🔧 Types of Function

📖 Definition

A **function** is a self-contained block of code that performs one specific task and can be reused wherever it's needed, instead of writing the same logic again and again. Every C program has at least one function — `main()` — and can have as many more as needed.

📁 Category 1 — Based on Who Defines It

Every function used in a C program falls into one of these two groups:

Type	Also Called	Who Writes It	Examples
Library Function	Built-in / Standard function	Already written for you, inside C's standard library	<code>printf()</code> , <code>scanf()</code> , <code>strlen()</code> , <code>sqrt()</code>
User-Defined Function	Custom function	Written by the programmer, for the specific program	<code>add()</code> , <code>factorial()</code> , <code>showMenu()</code>

To use a library function you must `#include` the header file that declares it — e.g. `<math.h>` for `sqrt()` , or `<string.h>` for `strlen()` .

✂ Category 2 — Based on Arguments & Return Value

This is the classification every beginner must know cold — a user-defined function always falls into exactly one of these four types:

#	Type	Takes Argument?	Returns Value?	Skeleton
1	No argument, no return value	✗ No	✗ No	<code>void greet() {...}</code>
2	No argument, with return value	✗ No	✓ Yes	<code>int getNumber() {...}</code>
3	With argument, no return value	✓ Yes	✗ No	<code>void greet(char n[]) {...}</code>
4	With argument, with return value	✓ Yes	✓ Yes	<code>int add(int a,int b){...}</code>

📄 Example Program — All Four Types Together

```
#include <stdio.h>
// Type 1: No argument, no return value
void greet() { printf("Hello, welcome!\n"); }
// Type 2: No argument, with return value
int getNumber() { return 5; }
// Type 3: With argument, no return value
void greetByName(char name[]) { printf("Hello, %s!\n", name); }
// Type 4: With argument, with return value
int add(int a, int b) { return a + b; }

int main() {
    greet();
    int num = getNumber();
    printf("Number = %d\n", num);
    greetByName("Riya");
    int sum = add(4, 5);
    printf("Sum = %d\n", sum);
    return 0;
}
```

▶ Output

```
Hello, welcome!
Number = 5
Hello, Riya!
Sum = 9
```

📌 Note

- `void` simply means "returns nothing" — the function does its job and comes back empty-handed.
- A function can take multiple arguments, but it can only return **one** value directly (unless you use pointers).
- Choosing the right type is a design decision: does this task need input? Does the caller need a result back?

🎉 Fun Tip

The 4 types are a vending machine 🚪: no-coin/no-snack, no-coin/free-sample, coin/donation-box, coin/snack — that last one is the "normal" function!

Function Declaration (Prototype)

Definition

A **function declaration** (also called a **function prototype**) tells the compiler a function's name, return type, and the type of parameters it expects — **before** the function is actually called or defined. Think of it as a promise: *"This function exists somewhere in the program, and here's exactly what it looks like."*

Why Do We Need It?

C reads a source file from top to bottom. If a function is **called** before the compiler has seen it even once, the compiler has no way to check whether the call is valid — right number of arguments? right types? right return type? Placing a declaration near the top of the file (usually just after the `#include` lines) solves this, letting you actually **define** the function anywhere in the file — even after `main()` .

Syntax

```
return_type function_name(parameter_type1, parameter_type2, ...);
```

Examples of valid declarations:

```
int add(int, int);           // parameter NAMES are optional here
float calculateArea(float radius);
void printMessage(void);    // takes nothing, returns nothing
```

⚠ A declaration always ends with a **semicolon** and has **no body** — just the header line. Mixing this up with a *definition* (which has a `{ }` body and no semicolon) is one of the most common beginner mistakes — covered in detail on the next page.

Example Program — Declare First, Define After main()

```
#include <stdio.h>

int add(int a, int b);    // declaration (prototype)

int main() {
    int result = add(3, 4); // compiler already knows add()'s signature
    printf("Sum = %d\n", result);
    return 0;
}

int add(int a, int b) {    // definition — comes AFTER main()
    return a + b;
}
```

Output

```
Sum = 7
```

Note

- Parameter names in a declaration are optional, and can even be different from the ones used in the definition — only the **types** matter to the compiler.
- If you skip the declaration *and* also define the function after `main()` , most modern compilers will raise an error ("implicit declaration of function").
- Functions defined **before** `main()` technically don't need a separate declaration — the definition itself acts as the declaration.

Fun Tip

A declaration is a restaurant menu 🍽 — it tells you what's on offer and what you must order (arguments) and expect back (return value), without showing you the kitchen yet!

Function Definition

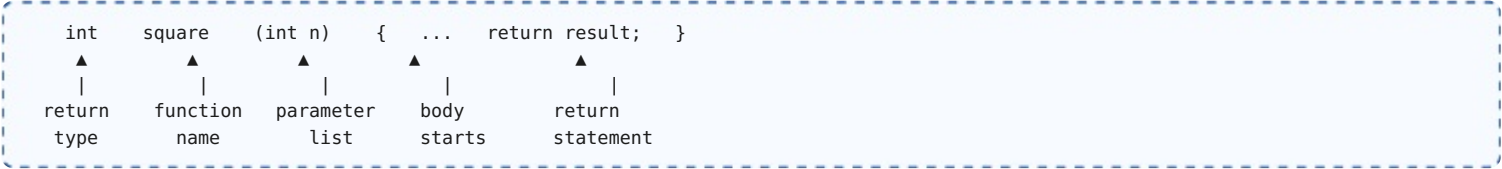
Definition

A **function definition** is the actual, complete block of code for a function. It includes the **header** (return type, name, parameter list) *and* the **body** — the set of statements inside `{ }` that run whenever the function is called.

Syntax

```
return_type function_name(parameter_list) {  
    // body: statements that do the work  
    return value; // only needed if return_type is not void  
}
```

Anatomy of a Function Definition



Part	Purpose
return_type	Data type of the value sent back to the caller (use <code>void</code> if nothing is returned)
function_name	The identifier used to call the function later
parameter_list	Variables that receive the values passed in during a call — can be empty
function body	The statements enclosed in <code>{ }</code> that actually perform the task
return statement	Sends a value back to the caller and immediately ends the function (skipped entirely for <code>void</code>)

Example Program

```
#include <stdio.h>  
  
int square(int n) {           // header: return type + name + parameter  
    int result = n * n;      // body  
    return result;          // return statement  
}  
  
int main() {  
    int x = 5;  
    printf("Square of %d = %d\n", x, square(x));  
    return 0;  
}
```

Output

```
Square of 5 = 25
```

⚠ Unlike a declaration, a definition **never** ends with a semicolon right after the parameter list — the `{ }` body follows immediately. A function can be **defined only once** in a program, but it can be **declared** (prototyped) any number of times.

Fun Tip

If a declaration is the restaurant menu, the definition is the actual recipe in the kitchen — that gets cooked every time someone places an order!

🔪 Function Call

📖 Definition

A **function call** is the statement that actually executes a function. It tells the program: *"jump to this function, run its code, and — if it returns something — come back with a result."*

🔗 Syntax

```
function_name(argument_list);
```

🔗 How Control Flows During a Call

```
main() is running
  |
  | addTen(x);   ► CALL: control jumps into addTen()
  v
addTen() runs its body completely
  |
  | function ends (or hits "return")
  v
control jumps back into main(), right after the call
```

🔗 Call by Value (C's Default Behaviour)

When you call a function normally, C copies the value of each argument into the function's parameters. The function works only on this **copy** — so any changes made inside the function have no effect on the original variable back in the caller.

📄 Example Program

```
#include <stdio.h>

void addTen(int n) {
    n = n + 10;
    printf("Inside function: n = %d\n", n);
}

int main() {
    int x = 5;
    addTen(x);    // function call - x is COPIED into n
    printf("Inside main: x = %d\n", x);    // x is unchanged!
    return 0;
}
```

► Output

```
Inside function: n = 15
Inside main: x = 5
```

📋 Step-by-Step: What Actually Happens

1. `x` is created in `main()` and set to 5.
2. `addTen(x)` is called — the **value** 5 is copied into the new local variable `n`.
3. Inside the function, `n` becomes 15 — but this only changes the copy, not `x`.
4. The function ends; `n` is destroyed. Control returns to `main()`, where `x` is still 5.

📌 Note

- This is exactly why, in the Pointers unit, you learned to pass an **address** (`&x`) instead of the value itself — that technique is called **call by reference**, and it's the way to let a function modify the caller's original variable.
- Every plain function call in C — unless you deliberately pass a pointer — is call by value.

💡 Fun Tip

Call by value = handing over a photocopy of your ID card 📄 . **Call by reference** = handing over the real card (via its address) 🗂️ !

Recursive Function

Definition

A **recursive function** is a function that calls **itself** — directly or indirectly — in order to solve a problem by breaking it down into smaller sub-problems of exactly the same type, until the sub-problem becomes simple enough to answer directly.

The Two Essential Parts

Part	Purpose
Base Case	The condition that stops the recursion. Without it, the function would call itself forever.
Recursive Case	The part where the function calls itself again, with a smaller/simpler version of the original problem.

⚠ Every recursive function **must** have a base case that is actually reachable. Forgetting it — or writing one that never gets hit — causes **infinite recursion**, which crashes the program with a **stack overflow**.

Example Program — Factorial Using Recursion

```
#include <stdio.h>

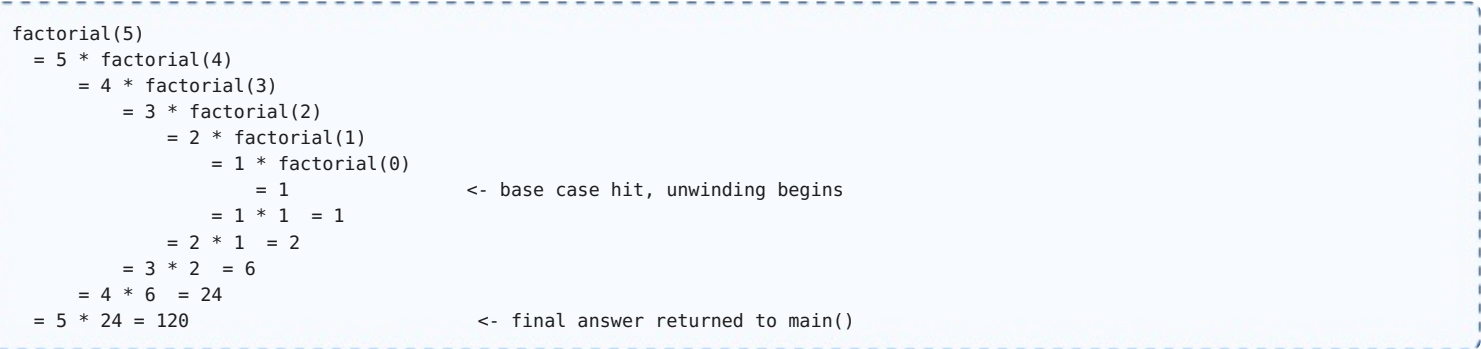
int factorial(int n) {
    if (n == 0)           // base case
        return 1;
    else
        return n * factorial(n - 1); // recursive case
}

int main() {
    printf("Factorial of 5 = %d\n", factorial(5));
    return 0;
}
```

Output

Factorial of 5 = 120

Visualizing the Calls — "Winding" Then "Unwinding"



Each call waits (paused, on the "call stack") for the one below it to finish before it can compute its own answer — that's why the calls only start multiplying out once the base case is reached.

Recursion vs Iteration (Loop)

Aspect	Recursion	Iteration (Loop)
How it repeats	Function calls itself	<code>for</code> / <code>while</code> repeats a block
Memory use	Higher — each call uses stack space	Lower — no extra function calls
Risk	Stack overflow if base case is missing/unreachable	Infinite loop if the exit condition is wrong
Best suited for	Naturally recursive problems (factorial, trees, Fibonacci)	Simple, repetitive counting/processing tasks

Note

- Any recursive solution can technically be rewritten using a loop, and vice versa — recursion is often chosen for how naturally it expresses the problem, not because it's the only way.
- Each recursive call gets its **own** copy of local variables (like `n`) — they don't overwrite each other, which is exactly why the "unwinding" works correctly.

Fun Tip

Recursion is a Matryoshka doll 🪆 — each doll (call) contains a smaller version of itself, until you reach the tiniest doll (base case) that won't open any further!